



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА — Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий

Кафедра вычислительной техники

ОТЧЁТ ПО ПРАКТИЧЕСКИМ РАБОТАМ № 1–2

по дисциплине

«Программирование нейропроцессоров»

*Тема: Системная и RTL-модели бинарного нейросетевого классификатора для
детектирования падений*

Студенты группы ИКБО-40-23:

Рябец М.Ф., Головин А.С., Гришин И.Д.

(Фамилия И. О. студента)

Руководитель работы:

Тарасов И.Е.

(Фамилия И. О. руководителя)

Москва 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
1 ПОСТАНОВКА ЗАДАЧИ И ДАТАСЕТ SISFALL.....	5
2 ИЗВЛЕЧЕНИЕ ПРИЗНАКОВ И ФОРМИРОВАНИЕ ВЫБОРОК.....	7
3 АРХИТЕКТУРА СЕТИ И КВАНТОВАНИЕ.....	9
4 ОБУЧЕНИЕ И ПРОГРАММНАЯ РЕАЛИЗАЦИЯ.....	11
5 РЕЗУЛЬТАТЫ И ЭТАЛОННЫЙ ОТКЛИК.....	13
6 ПОСТАНОВКА ЗАДАЧИ ПРАКТИЧЕСКОЙ РАБОТЫ № 2.....	16
7 РЕАЛИЗАЦИЯ RTL-МОДЕЛИ.....	17
8 СИМУЛЯЦИЯ И СВЕРКА С ЭТАЛОНОМ.....	18
ЗАКЛЮЧЕНИЕ.....	19
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	20
ПРИЛОЖЕНИЕ А КЛЮЧЕВЫЕ ФРАГМЕНТЫ ПРОГРАММЫ.....	21
ПРИЛОЖЕНИЕ Б ФРАГМЕНТЫ RTL-МОДЕЛИ.....	26
ПРИЛОЖЕНИЕ В ПОДТВЕРЖДЕНИЕ ПРОВЕРКИ В VIVADO.....	27

ВВЕДЕНИЕ

Падения — одна из основных причин травматизма у пожилых людей, при этом задержка помощи после падения существенно повышает риск осложнений. Носимый инерциальный датчик (акселерометр и гироскоп) позволяет детектировать падение автоматически, но такая система полезна только при работе в реальном времени на энергоэффективном устройстве — то есть на специализированной аппаратной логике, а не на универсальном процессоре с плавающей точкой.

Цель практических работ № 1–2 — построить программную эталонную модель нейроускорителя, затем разработать её RTL-эквивалент на Verilog и сверить результаты симуляции с эталонным откликом. В практической работе № 3 эта RTL-модель станет основой проекта для ПЛИС.

Для выполнения работы выбран вариант задания 2 (тема, предложенная студентами): детектирование падения человека — бинарная классификация «падение / не падение» по пяти признакам, извлечённым из данных акселерометра и гироскопа. Поскольку итоговая цель проекта — аппаратная реализация на ПЛИС, архитектура нейросети выбрана бинарной (Binary Neural Network): скрытый и выходной слои используют веса $\{-1, +1\}$ и вычисляются операциями XNOR и popcount вместо умножения, что убирает умножители из наиболее массовой по числу операций части схемы.

Для достижения цели поставлены следующие задачи:

- получить и разобрать открытый датасет SisFall, реализовать перевод сырых АЦП-кодов датчиков в физические единицы;
- извлечь из сигналов 5 признаков падения и сформировать обучающую и тестовую выборки без утечки данных между испытуемыми;
- спроектировать бинарную нейросеть и обучить её методом BinaryConnect / Straight-Through Estimator на NumPy;
- получить полностью целочисленную (int8, XNOR + popcount) эталонную реализацию сети и оценить потери точности от квантования;

- экспортировать эталонный отклик и веса в формате, готовом к использованию в RTL-тестбенче практической работы № 2.
- реализовать целочисленный классификатор $5 \rightarrow 8 \rightarrow 1$ на Verilog с загрузкой весов из .mem-файлов;
- выполнить симуляцию RTL-модели и проверить входы, промежуточные состояния и выходы по эталонным векторам.

1 ПОСТАНОВКА ЗАДАЧИ И DATASET SISFALL

1.1 Требования к системной модели и эталонному отклику

Согласно методическому указанию, результатом практической работы № 1 должна быть системная модель нейроускорителя, реализованная на языке программирования высокого уровня, и эталонный отклик — контрольные пары «вход — выход», по которым в дальнейшем верифицируется RTL-модель (практическая работа № 2) и прототип на ПЛИС (практическая работа № 3). Отсюда вытекает ключевое техническое требование: модель должна допускать точную целочисленную реализацию (без плавающей точки), иначе эталонный отклик было бы невозможно воспроизвести бит в бит на RTL.

1.2 Датасет SisFall

Использован открытый датасет SisFall (A. Sucerquia, J.D. López, J.F. Vargas-Bonilla, SISTEMIC, Universidad de Antioquia, 2016) — один из основных общедоступных наборов данных для задач детектирования падений. Официальный сайт проекта на момент выполнения работы недоступен; архив получен без изменений из зеркала на GitHub (github.com/BIng2325/SisFall).

Датасет содержит записи 38 испытуемых: 23 взрослых (SA01–SA23, 19–30 лет) и 15 пожилых (SE01–SE15, 60–75 лет); 19 видов повседневной активности (коды D01–D19) и 15 видов падений (коды F01–F15), всего 4506 файлов. Данные получены тремя датчиками с частотой опроса 200 Гц: акселерометром ADXL345 (диапазон $\pm 16g$, разрешение 13 бит), гироскопом ITG3200 (диапазон ± 2000 °/с, разрешение 16 бит) и вторым акселерометром MMA8451Q (в проекте не используется). Перевод «сырых» АЦП-кодов AD в физические единицы выполняется по формуле, приведённой в Readme датасета:

$$x_{phys} = \frac{2 \cdot Range}{2^{Resolution}} \cdot AD \quad (1.1)$$

Пожилые испытуемые (кроме SE06) падения не выполняли по медицинским показаниям — это особенность протокола сбора данных, учтённая при разбиении на обучающую и тестовую выборки (раздел 2.2).

2 ИЗВЛЕЧЕНИЕ ПРИЗНАКОВ И ФОРМИРОВАНИЕ ВЫБОРОК

2.1 Формулы пяти признаков

Согласно постановке задачи проекта, для классификации используется 5 признаков. Один образец (одно окно анализа) строится вокруг локального пика модуля результирующего ускорения записи — физически обоснованного момента удара при падении или пиковой фазы резкого движения при обычной активности. Для коротких испытаний (12–25 с) берётся одно окно на весь файл; длинные записи ходьбы и бега (100 с) нарезаются на скользящие окна по 4 с с шагом 2 с.

Признак 1 — модуль результирующего ускорения в момент пика:

$$|a(t_p)| = \sqrt{a_x^2 + a_y^2 + a_z^2} \quad (2.1)$$

Признак 3 (пиковая угловая скорость в окрестности пика) вычисляется аналогичной формулой по проекциям гироскопа gx , gy , gz . Признак 2 — угол наклона тела относительно опорной (начальной) ориентации, где a_{ref} — средний вектор ускорения за первые 0,5 с записи (протокол начинается с неподвижного состояния):

$$tilt = \arccos\left(\frac{a(t_p) \cdot a_{ref}}{|a(t_p)| \cdot |a_{ref}|}\right) \quad (2.2)$$

Признак 4 — $jerk$ (скорость изменения ускорения), максимум модуля конечной разности в окрестности пика $\pm 0,25$ с, $f_s = 200$ Гц — частота дискретизации:

$$jerk = \max(f_s \cdot ||a(n) - a(n - 1)||) \quad (2.3)$$

Признак 5 — дисперсия результирующего ускорения на интервале $[t_p+0,5с, t_p+1,5с]$ после пика: близкая к нулю дисперсия означает, что человек лежит неподвижно (настоящее падение), высокая — что движение продолжается (сел, наклонился). Разделение классов по всем 5 признакам

показано на рисунке 2.1: все признаки статистически различимы между классами.

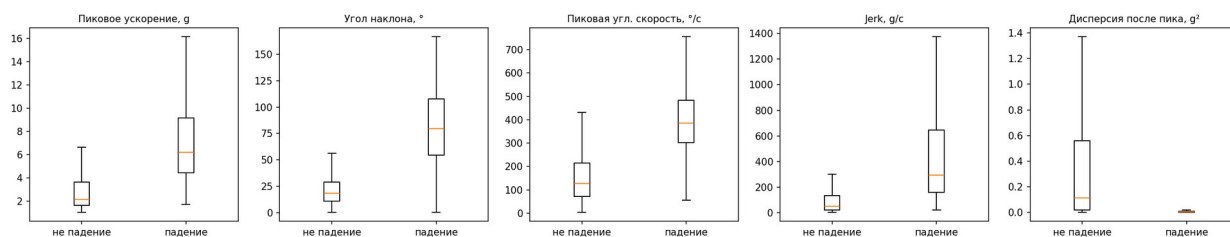


Рисунок 2.1 — Распределение признаков по классам (боксплоты)

Метка окна наследуется от типа исходного файла: любое окно из файла с кодом F* получает метку 1 (падение), из файла с кодом D* — метку 0 (обычная активность) — стандартный для SisFall-подобных исследований способ разметки, так как каждое испытание в протоколе посвящено одной контролируемой активности.

2.2 Разметка и разбиение на train/test

Итоговый набор признаков — 11839 окон (1798 падений, 15,2 %; 10041 не-падений, 84,8 % — ожидаемый дисбаланс классов, так как протокол предусматривает больше сценариев обычной активности, чем падений). Разбиение на обучающую и тестовую выборки выполнено по испытуемым (subject-wise split), а не по отдельным окнам: несколько окон одного человека похожи друг на друга, и попадание одного субъекта одновременно в train и test зависило бы оценку качества. В тест выделены испытуемые SA20, SA21, SA22, SA23, SE12, SE13, SE14, SE15 (8 из 38, ~21 %), полностью исключённые из обучения (таблица 2.1).

Таблица 2.1 — Разбиение датасета на обучающую и тестовую выборки

Выборка	Испытуемых	Окон	Падений	Не падений
Train	30	9508	1500	8008
Test	8	2331	298	2033

3 АРХИТЕКТУРА СЕТИ И КВАНТОВАНИЕ

3.1 Бинарная нейросеть $5 \rightarrow 8 \rightarrow 1$

Сеть построена из двух слоёв. Слой А (кодировщик, fixed-point) линейно комбинирует 5 квантованных признаков в 8 промежуточных сигналов и бинаризует их функцией `sign` — эта операция остаётся в целочисленной ($\text{int8} \times \text{int8} \rightarrow \text{int32}$) арифметике, но умножителей здесь всего 40 (5×8), и разрядность мала. Слой В (классификатор, истинно бинарный) линейно комбинирует 8 бинарных сигналов весами $\in \{-1, +1\}$ и сравнивается с порогом: скалярное произведение двух ± 1 -векторов вычисляется без единого умножителя через тождество `XNOR + popcount`:

$$\text{dot}(a, w) = 2 \cdot \text{popcount}(\text{XNOR}(a, w)) - N \quad (3.1)$$

где $N = 8$ — число нейронов слоя А. Тождество проверено численно на 2000 случайных парах ± 1 -векторов разной длины (см. листинг А.9 и `tests/test_golden_model.py`) — совпадение полное. Именно эта граница (fixed-point препроцессор $\rightarrow$ истинно бинарный классификатор) выбрана как объём «нейроускорителя», переносимого на ПЛИС в практических работах № 2–3: слой В почти не требует логики (XNOR-вентили, сумматор единиц, компаратор), а слой А — простое MAC малой разрядности. Всего обучаемых параметров: $40 (W_A, \text{int8}) + 8 (b_A, \text{int32}) + 8 (W_B, 1 \text{ бит}) + 1 (b_B, \text{int32}) = 57$ скалярных величин.

3.2 Fixed-point квантование

Квантование признаков — симметричное, без нулевой точки:

$$q = \text{clip}\left(\text{round}\left(\frac{x}{\text{scale}}\right), -128, 127\right) \quad (3.2)$$

Масштаб `scale` для каждого из 5 признаков калибруется по обучающей выборке отдельно (диапазоны физически разные: `g`, градусы, $^{\circ}/\text{с}$, $\text{g}/\text{с}$, g^2) — берётся 99,5-й перцентиль модуля значения, а не максимум, чтобы редкие

выбросы не «съедали» динамический диапазон 8 бит. Веса слоя A квантуются одним общим масштабом на всю матрицу $w_scale_A = \max(|W_A|)/127$, что корректно именно потому, что входы уже приведены к единому нормированному диапазону на предыдущем шаге. Смещение слоя квантуется в единицах того же аккумулятора ($b_A_i32 = \text{round}(b_A / (w_scale_A/127))$), что сохраняет знак преактивации z — а только знак и используется, так как за слоем A следует $\text{sign}()$.

4 ОБУЧЕНИЕ И ПРОГРАММНАЯ РЕАЛИЗАЦИЯ

4.1 BinaryConnect / Straight-Through Estimator

Прямое распространение сигнала через $\text{sign}()$ не дифференцируемо, поэтому обучение выполнено методом BinaryConnect / Straight-Through Estimator (Courbariaux et al., 2016) — реализовано вручную на NumPy без сторонних ML-библиотек (PyTorch недоступен в окружении). Хранятся вещественные «теневые» веса W_A , W_B ; на прямом проходе W_B и активации после слоя A бинаризируются знаком; на обратном проходе производная $\text{sign}(t)$ заменяется производной hard tanh :

$$\frac{d \cdot \text{sign}(t)}{dt} \approx 1, |t| \leq 1; 0, \text{ иначе} \quad (4.1)$$

После каждого шага теневые веса W_B клипуются в $[-1, 1]$ (классический приём BinaryConnect, иначе веса «убегают» и штраф STE перестаёт работать корректно). Ключевые фрагменты прямого и обратного прохода приведены в листингах А.5 и А.6.

4.2 Параметры обучения и сходимость

Параметры обучения: оптимизатор Adam (ручная реализация, $\text{lr} = 0,05$), 300 эпох, размер батча 64, функция потерь — взвешенная бинарная кросс-энтропия (вес положительного класса $\approx 5,34 = 8008/1500$, чтобы скомпенсировать дисбаланс классов). Кривая обучения показана на рисунке 4.1.

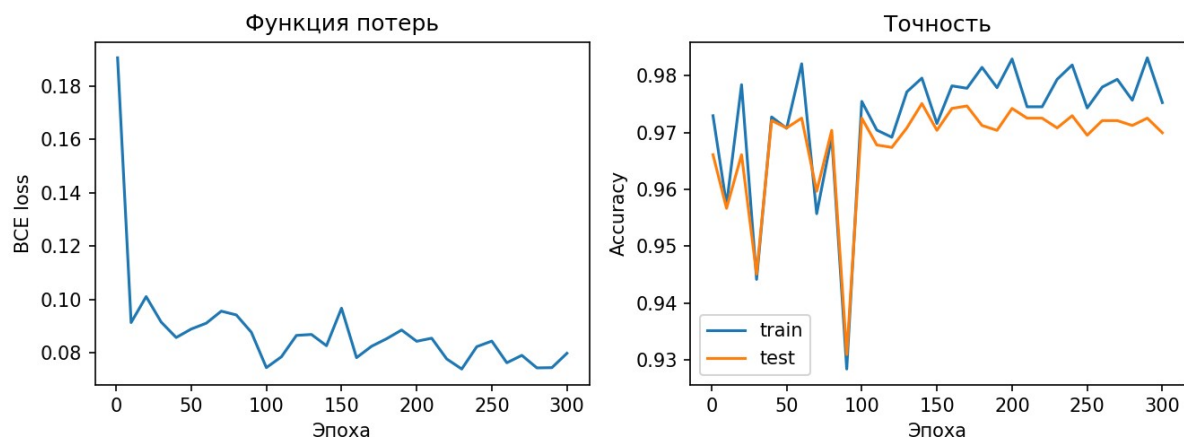


Рисунок 4.1 — Функция потерь и точность на train/test по эпохам

5 РЕЗУЛЬТАТЫ И ЭТАЛОННЫЙ ОТКЛИК

5.1 Метрики качества

Оценка выполнена на тестовой выборке (2331 окно, 8 испытуемых, не встречавшихся при обучении) для двух версий модели: float-модель (вещественные веса, `sign()` как при обучении) и int8 golden-модель (полностью целочисленная реализация, результат которой и есть эталонный отклик, таблица 5.1).

Таблица 5.1 — Метрики качества на тестовой выборке

Метрика	float-модель	int8 golden-модель
Accuracy	97,0 %	97,0 %
Precision	84,3 %	84,5 %
Recall	94,0 %	93,6 %
F1	0,8889	0,8885
TP / TN / FP / FN	280 / 1981 / 52 / 18	279 / 1982 / 51 / 19

Совпадение решений float- и int8-модели: 99,9 % (расходятся всего 2 из 2331 примеров, в обоих случаях — приграничные значения около порога $y \approx 0$), что подтверждает: квантование до int8/бинарных весов практически не ухудшает качество (рисунок 5.1).

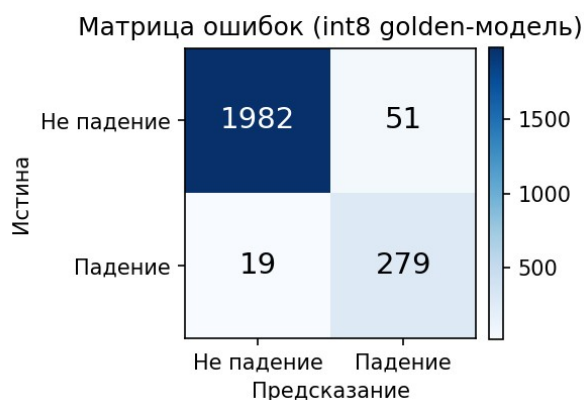


Рисунок 5.1 — Матрица ошибок, int8 эталонная модель

Recall 93,6 % означает, что модель пропускает около 6,4 % реальных падений на тесте — для системы безопасности это стоит улучшать в первую очередь (например, решением по нескольким последовательным окнам), что выходит за рамки практики 1. Precision 84,5 % отражает конкретную особенность выбранных признаков: часть резких, но безопасных движений (быстрая посадка, прыжок) даёт схожий по «остроте» профиль ускорения и jerk, что и настоящее падение — известная проблема чисто акселерометрических детекторов падения, отмеченная и в исходной статье по SisFall.

5.2 Эталонный отклик для практической работы № 2

Результаты практической работы № 1, используемые для верификации RTL-модели в практической работе № 2, размещены в каталоге `weights/` репозитория:

- `weights/golden_vectors_full.csv` — полный тестовый набор из 2331 вектора: квантованные входы, восьмибитный код слоя A, сумма слоя B и выход;
- `weights/tb_inputs_64.mem` и `weights/tb_expected_64.mem` — 64 отобранных пары «вход — эталонный выход» для начальной симуляции;
- `weights/weights_layerA_W.mem`, `weights_layerA_b.mem`, `weights_layerB_W.mem`, `weights_layerB_b.mem` — квантованные веса и смещения для \$readmemh;
- `weights/README.md` — точный порядок байт, битов и строк в экспортированных файлах.

Корректность экспортированных файлов проверена автоматическим тестом: декодирование `tb_inputs_64.mem` обратно в `int8` и прогон через целочисленную модель на 100 % воспроизводит `tb_expected_64.mem`.

5.3 Границы объёма проекта на практические работы № 2–3

«Нейроускоритель», переносимый на ПЛИС, ограничен классификатором (слой A + слой B) — принимает 5 уже готовых квантованных признаков и выдаёт 1 бит класса. Извлечение признаков из сырого потока датчиков (sqrt, arccos, поиск пика, дисперсия) остаётся программным препроцессингом и тестовыми векторами, а не переносится в RTL: эти операции существенно сложнее для аппаратной реализации (нужны CORDIC или LUT-аппроксимации), тогда как сам классификатор специально спроектирован без единого умножителя во втором слое. Такое разделение соответствует практике реальных NPU IP-блоков, где сенсорный DSP-фронтенд отделён от ядра нейроускорителя.

6 ПОСТАНОВКА ЗАДАЧИ ПРАКТИЧЕСКОЙ РАБОТЫ

№ 2

6.1 Цель и границы RTL-модели

Требование практической работы № 2 — реализовать на Verilog RTL-модель нейроускорителя, созданного в первой работе, и методом симуляции сверить её отклик с программным эталоном. Переносится только классификатор: пять уже рассчитанных и квантованных признаков подаются на вход. Поиск пика, вычисление угла, jerk и дисперсии остаются программной подготовкой данных. Такой же вычислительный контур выбран для эталонной модели в разделе 3.

6.2 Интерфейс и формат данных

Входные признаки передаются одним 40-битным словом {x0, x1, x2, x3, x4}; каждый байт является знаковым числом int8 в дополнительном коде. Порядок соответствует файлам weights/. Результат регистрируется на фронте тактового сигнала при valid_in=1. Сигнал valid_out показывает, что выходной бит относится к принятому входу. Сброс rst_n синхронный, активный низким уровнем.

Таблица 6.1 — Интерфейс RTL-классификатора

Сигнал	Бит	Назначение
clk	1	Тактовый сигнал
rst_n	1	Синхронный сброс, активный 0
valid_in	1	На входе подготовлено слово признаков
input_features	40	Пять знаковых байтов x0...x4
valid_out	1	Зарегистрированный ответ действителен
fall_detected	1	1 — падение, 0 — обычная активность

Для воспроизводимости проект ориентирован на Vivado 2022.2 и кристалл xc7a100tcs324-1, указанный для курса. Tcl-сценарий rtl/create_vivado_project.tcl добавляет Verilog, тестбенч и файлы памяти в проект. Выбор кристалла не влияет на результаты поведенческой симуляции.

7 РЕАЛИЗАЦИЯ RTL-МОДЕЛИ

7.1 Слой A: знаковое умножение и накопление

Восемь нейронов слоя A вычисляют $z_h = b_A[h] + \sum_j x_j \cdot W_A[j,h]$ для $j=0\dots4$. Веса W_A загружаются в порядке $j \cdot 8 + h$. Операнды `int8` явно трактуются как знаковые; произведение хранится в 16 битах и расширяется до 32 бит перед сложением со смещением `int32`. Промежуточный бит `a_h` равен 1 тогда и только тогда, когда $z_h \geq 0$. Нулевой случай включён намеренно: так определён эталон в `model/src/golden_model.py`.

7.2 Слой B: XNOR и popcount

Восемь бинарных весов W_B сравниваются с битами `a_h` операцией XNOR. Четырёхбитный счётчик определяет число совпадений от 0 до 8. Итоговая сумма вычисляется как $y = 2 \cdot \text{popcount}(\text{XNOR}(a, W_B)) - 8 + b_B$; выход `fall_detected` равен 1 при $y \geq 0$. Эта формула бит в бит соответствует программной модели и не требует вещественной арифметики во время вывода.

7.3 Регистры, сброс и загрузка параметров

Комбинационная часть рассчитывает оба слоя для текущего входа. На ближайшем фронте `clk` при `valid_in=1` сохраняются выходной бит и `valid_out=1`; при отсутствии входа `valid_out=0`, а последний бит остаётся неизменным. Синхронный сброс обнуляет оба регистра. Веса и смещения загружаются из четырёх `.mem`-файлов через `$readmemh`. Их формат и порядок фиксированы в `weights/README.md`; переобучение сети для ПР2 не выполняется.

8 СИМУЛЯЦИЯ И СВЕРКА С ЭТАЛОНОМ

8.1 Методика верификации

Тестбенч `rtl/tb_bnn_classifier.v` подаёт вход до фронта `clk` и сверяет зарегистрированный выход и `valid_out` с эталоном. На полном наборе он сравнивает также `a_bits` и сумму `y`, что локализует ошибки по слоям. Отдельно проверяются сброс и снятие `valid_out`. Полные тестовые `.mem`-файлы строятся из `weights/golden_vectors_full.csv` сценарием `rtl/gen_full_tb.py`.

В Icarus Verilog 13.0 проверены 64 исходных, 2331 тестовых и 14 граничных векторов со знаковыми значениями -128 и 127 . Ожидаемый выход — решение программной модели, а не истинная метка `SisFall`.

8.2 Результаты

Таблица 8.1 — Результаты поведенческой симуляции

Набор	Число	Расхождения по выходу	Расхождения внутри слоёв
Исходный контрольный	64	0	Промежуточные значения не сравнивались
Полный тестовый	2331	0	0 по <code>a_bits</code> и <code>y</code>
Граничные значения <code>int8</code>	14	0	0 по <code>a_bits</code> и <code>y</code>
Vivado XSim, полный набор	2331	0	0 по <code>a_bits</code> и <code>y</code>

Например, для $x=[14,16,27,2,0]$ модель и RTL дали `a_bits=10100001`, `y=-5`, `fall_detected=0`. Во всех наборах расхождений не обнаружено. Это подтверждает функциональную эквивалентность RTL целочисленному эталону на проверенных входах.

В Vivado 2022.2 создан проект `bnn_pr2` для `xc7a100tcsg324-1`. Поведенческая симуляция XSim завершилась сообщением `PASS: 2331 vectors`. Временные диаграммы, результат синтеза и схема приведены в приложении В. Синтез успешно завершился: использовано 629 Slice LUT, 2 Slice Register, 45 Bonded IOB и 1 BUFGCTRL. Оценка ресурсов относится только к синтезу; размещение, трассировка и анализ таймингов не выполнялись.

ЗАКЛЮЧЕНИЕ

В практических работах № 1–2 создана программная и RTL-модель бинарного классификатора падений $5 \rightarrow 8 \rightarrow 1$. Программная модель обучена на признаках SisFall и дала на тестовой выборке 97,0 % accuracy, 84,5 % precision и 93,6 % recall. RTL-модель повторяет целочисленный тракт: знаковые int8-умножения, 32-битное накопление, бинаризацию, XNOR и popcount. В поведенческой симуляции Icarus Verilog получено полное совпадение с эталоном на 64 контрольных, 2331 тестовых и 14 граничных входах; на полном наборе совпали также промежуточные состояния обоих слоёв. В Vivado 2022.2 создан проект для xc7a100tcsg324-1, симуляция XSim прошла 2331 вектор без расхождений, синтез завершился успешно и получена логическая схема. Таким образом, системная модель и её функционально проверенный RTL-эквивалент готовы к следующему этапу работы.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Практическая работа № 1 «Разработка системной модели нейроускорителя»: задание по дисциплине «Программирование нейропроцессоров». — М.: РТУ МИРЭА, 2026. — 2 с.
2. Sucerquia A., López J.D., Vargas-Bonilla J.F. SisFall: A Fall and Movement Dataset // Sensors. — 2017. — Vol. 17, № 1. — Art. 198.
3. Courbariaux M., Bengio Y., David J.-P. BinaryConnect: Training Deep Neural Networks with Binary Weights during Propagations // Advances in Neural Information Processing Systems 28 (NeurIPS 2015). — 2015. — P. 3123–3131.
4. Rastegari M., Ordonez V., Redmon J., Farhadi A. XNOR-Net: ImageNet Classification Using Binary Convolutional Neural Networks // European Conference on Computer Vision (ECCV 2016). — 2016. — P. 525–542.
5. Правила оформления курсовой работы (проекта), ВКР и письменных работ бакалавра и магистра. — М.: РТУ МИРЭА, Институт информационных технологий, 2023. — 24 с.
6. NumPy Developers. NumPy Reference. Version 2.4. — 2026. — URL: <https://numpy.org/doc/stable/reference/> (дата обращения: 28.09.2026).
7. SisFall dataset mirror. — URL: <https://github.com/BIng2325/SisFall> (дата обращения: 28.09.2026).
8. AMD. Vivado Design Suite User Guide: Logic Simulation (UG900), version 2022.2. — URL: <https://docs.amd.com/r/2022.2-English/ug900-vivado-logic-simulation/> (дата обращения: 03.10.2026).
9. AMD. Vivado Design Suite User Guide: Synthesis (UG901), version 2022.2. — URL: <https://docs.amd.com/r/2022.2-English/ug901-vivado-synthesis/> (дата обращения: 03.10.2026).

Приложение А

Ключевые фрагменты программы

Полный исходный код программной модели находится в каталоге `model/src/` репозитория. Ниже приведены фрагменты, определяющие вычислительную часть работы.

Листинг А.1 — Перевод АЦП-кодов SisFall в физические единицы

```
def load_recording(path: Path) -> Recording:
    code, subject, trial = parse_filename(path)
    raw_text = path.read_text().replace(";", "")
    raw = np.genfromtxt(raw_text.splitlines(), delimiter=",",
dtype=np.float64)
    if raw.ndim == 1:
        raw = raw.reshape(1, -1)
    if raw.shape[1] != 9:
        raise ValueError(f"{path}: expected 9 columns, got {raw.shape[1]}")

    acc_g = raw[:, 0:3] * ACC_SCALE
    gyro_dps = raw[:, 3:6] * GYRO_SCALE

    return Recording(
        path=path,
        code=code,
        subject=subject,
        trial=trial,
        is_fall=code in FALL_CODES,
        acc_g=acc_g,
        gyro_dps=gyro_dps,
    )
```

```
def _extract_one_window(rec: Recording, res_acc: np.ndarray, res_gyro:
np.ndarray,
                        ref_vec: np.ndarray, search_lo: int, search_hi: int)
-> FeatureSample | None:
    n = rec.n_samples
    search_hi = min(search_hi, n)
    if search_hi - search_lo < 10:
        return None

    local_peak = int(np.argmax(res_acc[search_lo:search_hi]))
    peak_idx = search_lo + local_peak

    peak_acc_g = float(res_acc[peak_idx])

    a_peak = rec.acc_g[peak_idx]
    cos_theta = np.dot(a_peak, ref_vec) / (np.linalg.norm(a_peak) *
np.linalg.norm(ref_vec) + 1e-9)
    tilt_deg = float(np.degrees(np.arccos(np.clip(cos_theta, -1.0, 1.0))))

    gyro_nb = int(GYRO_NEIGHBORHOOD_S * FS)
    g_lo, g_hi = max(0, peak_idx - gyro_nb), min(n, peak_idx + gyro_nb)
    peak_gyro_dps = float(np.max(res_gyro[g_lo:g_hi]))

    jerk_nb = int(JERK_NEIGHBORHOOD_S * FS)
    j_lo, j_hi = max(1, peak_idx - jerk_nb), min(n, peak_idx + jerk_nb)
    if j_hi - j_lo < 2:
        jerk_g_per_s = 0.0
    else:
        jerk_series = np.abs(np.diff(res_acc[j_lo - 1:j_hi])) * FS
        jerk_g_per_s = float(np.max(jerk_series))

    p_lo = peak_idx + int(POST_PEAK_START_S * FS)
    p_hi = peak_idx + int(POST_PEAK_END_S * FS)
    p_hi = min(p_hi, n)
    if p_hi - p_lo < MIN_POST_PEAK_SAMPLES:
        return None
    post_peak_var_g2 = float(np.var(res_acc[p_lo:p_hi]))

    features = np.array([peak_acc_g, tilt_deg, peak_gyro_dps, jerk_g_per_s,
post_peak_var_g2])
    return FeatureSample(
        features=features,
        label=int(rec.is_fall),
        subject=rec.subject,
        code=rec.code,
        trial=rec.trial,
        peak_idx=peak_idx,
    )
```

Листинг A.3 — Обход записей и построение выборок с меткой

```
def extract_samples(rec: Recording) -> list[FeatureSample]:
    n = rec.n_samples
    res_acc = _resultant(rec.acc_g)
    res_gyro = _resultant(rec.gyro_dps)
    calib_n = max(1, min(n, int(CALIBRATION_S * FS)))
    ref_vec = np.mean(rec.acc_g[:calib_n], axis=0)
    if np.linalg.norm(ref_vec) < 1e-6:
        ref_vec = np.array([0.0, 0.0, 1.0])

    samples: list[FeatureSample] = []
    if rec.code in LONG_TRIAL_CODES:
        start = 0
        while start < n:
            hi = min(start + WINDOW_N, n)
            s = _extract_one_window(rec, res_acc, res_gyro, ref_vec, start,
hi)
                if s is not None:
                    samples.append(s)
                if hi >= n:
                    break
                start += STRIDE_N
    else:
        s = _extract_one_window(rec, res_acc, res_gyro, ref_vec, 0, n)
        if s is not None:
            samples.append(s)
    return samples
```

Листинг A.4 — Калибровка масштаба int8-квантования признаков

```
def calibrate(x: np.ndarray, percentile: float = CALIBRATION_PERCENTILE) ->
QuantParams:
    """x: (N, C). Возвращает по одному масштабу на каждый из C столбцов."""
    x = np.atleast_2d(x)
    abs_max = np.percentile(np.abs(x), percentile, axis=0)
    abs_max = np.maximum(abs_max, 1e-8)
    scale = abs_max / INT8_MAX
    return QuantParams(scale=scale)
```

Листинг A.5 — Прямой проход BNN (бинаризация активаций и весов слоя B)

```
def forward(self, X: np.ndarray):
    z = X @ self.W_A + self.b_A          # (N, H)
    a = sign(z)                          # (N, H) in {-1,+1}
    Wb = sign(self.W_B)                  # (H,)   in {-1,+1}
    y = a @ Wb + self.b_B                # (N,)
    p = sigmoid(y)
    cache = dict(X=X, z=z, a=a, Wb=Wb, y=y)
    return p, cache
```

Листинг A.6 — Обратный проход BNN (STE-градиенты)

```
def backward(self, cache: dict, label: np.ndarray, sample_weight:
np.ndarray | None = None):
    X, z, a, Wb = cache["X"], cache["z"], cache["a"], cache["Wb"]
    N = X.shape[0]
    p = sigmoid(cache["y"])
    w = np.ones(N) if sample_weight is None else sample_weight
    w = w / w.sum() # нормировка, чтобы масштаб градиента не зависел от
размера батча

    dLdy = (p - label) * w # (N,) d(BCE)/dy при y =
логит
    dLdb_B = np.sum(dLdy)
    dLdWb = a.T @ dLdy # (H,)
    dLdW_B = dLdWb * (np.abs(self.W_B) <= 1.0) # STE-клип

    dLda = np.outer(dLdy, Wb) # (N, H)
    dLdz = dLda * (np.abs(z) <= 1.0) # STE-клип по
преактивации слоя A

    dLdW_A = X.T @ dLdz # (n_in, H)
    dLdb_A = np.sum(dLdz, axis=0)

    return dict(W_A=dLdW_A, b_A=dLdb_A, W_B=dLdW_B, b_B=dLdb_B)
```

Листинг A.7 — Квантование обученной модели в int8 / бинарные веса

```
def quantize_model(W_A: np.ndarray, b_A: np.ndarray, W_B: np.ndarray, b_B:
float,
                  in_scale: np.ndarray) -> GoldenWeights:
    w_scale_A = max(float(np.max(np.abs(W_A))) / INT8_MAX, 1e-8)
    W_A_i8 = np.clip(np.round(W_A / w_scale_A), INT8_MIN,
INT8_MAX).astype(np.int32)
    b_A_i32 = np.round(b_A / (w_scale_A / INT8_MAX)).astype(np.int64)

    W_B_sign = np.where(W_B >= 0, 1, -1)
    W_B_bit = (W_B_sign > 0).astype(np.int8)
    b_B_i32 = int(np.round(b_B))

    return GoldenWeights(
        in_scale=in_scale.astype(np.float64),
        W_A_i8=W_A_i8.astype(np.int32),
        b_A_i32=b_A_i32.astype(np.int64),
        w_scale_A=w_scale_A,
        W_B_bit=W_B_bit,
        b_B_i32=b_B_i32,
    )
```

Листинг A.8 — Целочисленный прямой проход эталонной модели

```
def forward_int(gw: GoldenWeights, X_i8: np.ndarray):
    """X_i8: (N, n_in) int. Возвращает (z_i32, a_bit, y_i32, label)."""
    X_i8 = np.atleast_2d(X_i8).astype(np.int64)
    z_i32 = X_i8 @ gw.W_A_i8.astype(np.int64) + gw.b_A_i32 # (N, H)
    a_bit = (z_i32 >= 0).astype(np.int8) # (N, H)

    y_i32 = xnor_popcount_dot(a_bit, np.broadcast_to(gw.W_B_bit, a_bit.shape))
+ gw.b_B_i32
    label = (y_i32 >= 0).astype(np.int32)
    return z_i32, a_bit, y_i32, label
```

Листинг А.9 — Тождество XNOR + popcount для ± 1 -векторов

```
def xnor_popcount_dot(a_bit: np.ndarray, w_bit: np.ndarray) -> np.ndarray:
    """dot(a_pm1, w_pm1) через XNOR + popcount, a_bit/w_bit in {0,1}."""
    xnor = 1 - (a_bit ^ w_bit)          # 1, если биты совпали (оба +1 или
оба -1)
    H = w_bit.shape[-1]
    popcount = np.sum(xnor, axis=-1)
    return 2 * popcount - H
```

Приложение Б

Ключевые фрагменты RTL-модели и тестбенча

Полные исходники находятся в rtl/bnn_classifier.v и rtl/tb_bnn_classifier.v.

Здесь показаны операции, определяющие совпадение с эталоном.

Листинг Б.1 — Знаковое накопление и бинаризация слоя A

```
accumulator = $signed(b_a[h]);
for (j = 0; j < 5; j = j + 1) begin
    product = $signed(input_features[39 - 8*j -: 8])
        * $signed(w_a[j*8 + h]);
    accumulator = accumulator + {{16{product[15]}}, product};
end
hidden_bits[h] = (accumulator >= 32'sd0);
```

Листинг Б.2 — XNOR, popcount и пороговое решение

```
for (h = 0; h < 8; h = h + 1)
    matching_bits = matching_bits + (hidden_bits[h] ^ w_b[h]);
score = ($signed({28'b0, matching_bits}) <<< 1)
    - 32'sd8 + $signed(b_b[0]);
prediction_next = (score >= 32'sd0);
```

Приложение В

Подтверждение проверки в Vivado 2022.2

Снимки экрана получены при запуске проекта bnn_pr2 для xc7a100tcsg324-1. Они фиксируют состав проекта, результат поведенческой симуляции, временные диаграммы и синтезированную схему с отчётом использования ресурсов.

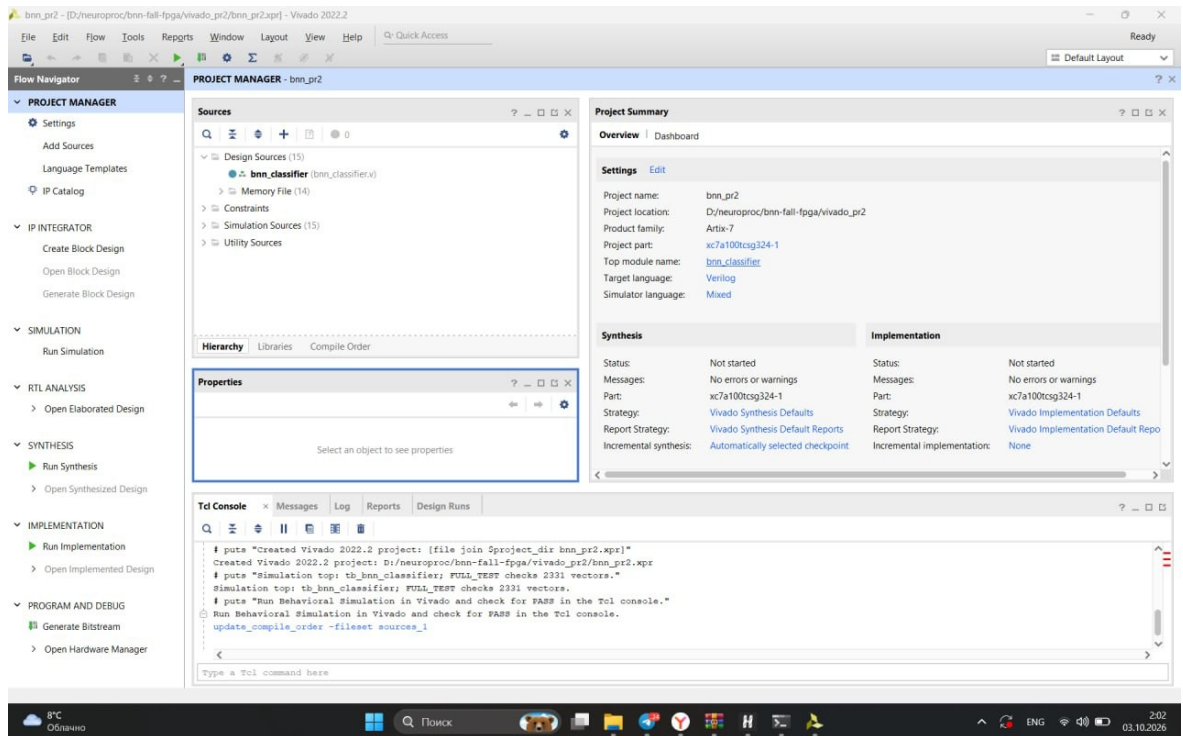


Рисунок В.1 — Проект bnn_pr2 и целевой кристалл

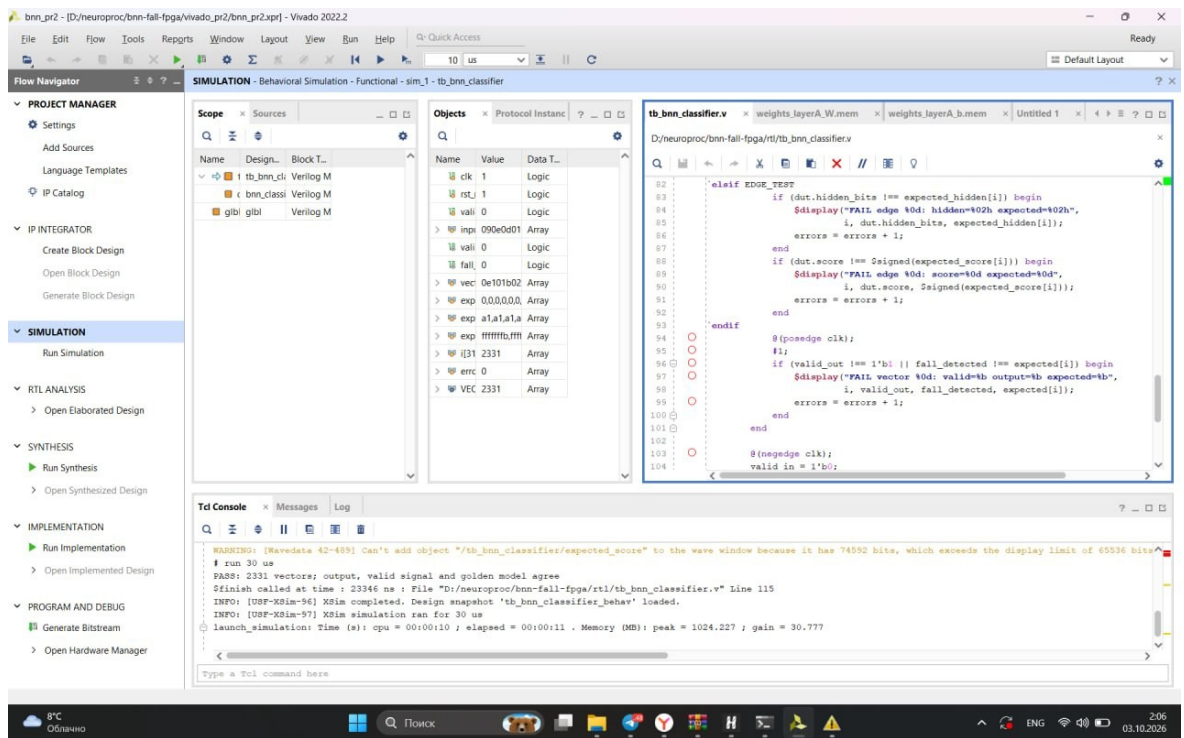


Рисунок В.2 — Успешная симуляция 2331 эталонного вектора

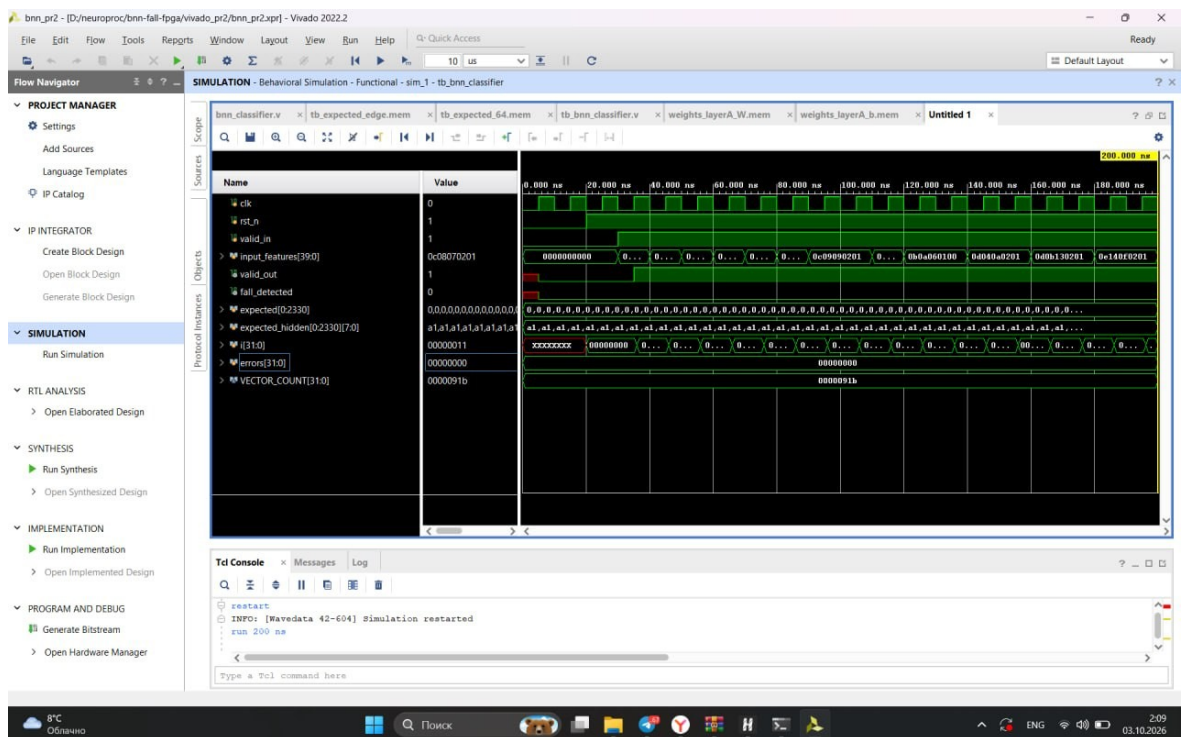


Рисунок В.3 — Временные диаграммы входов и выходов RTL-модели

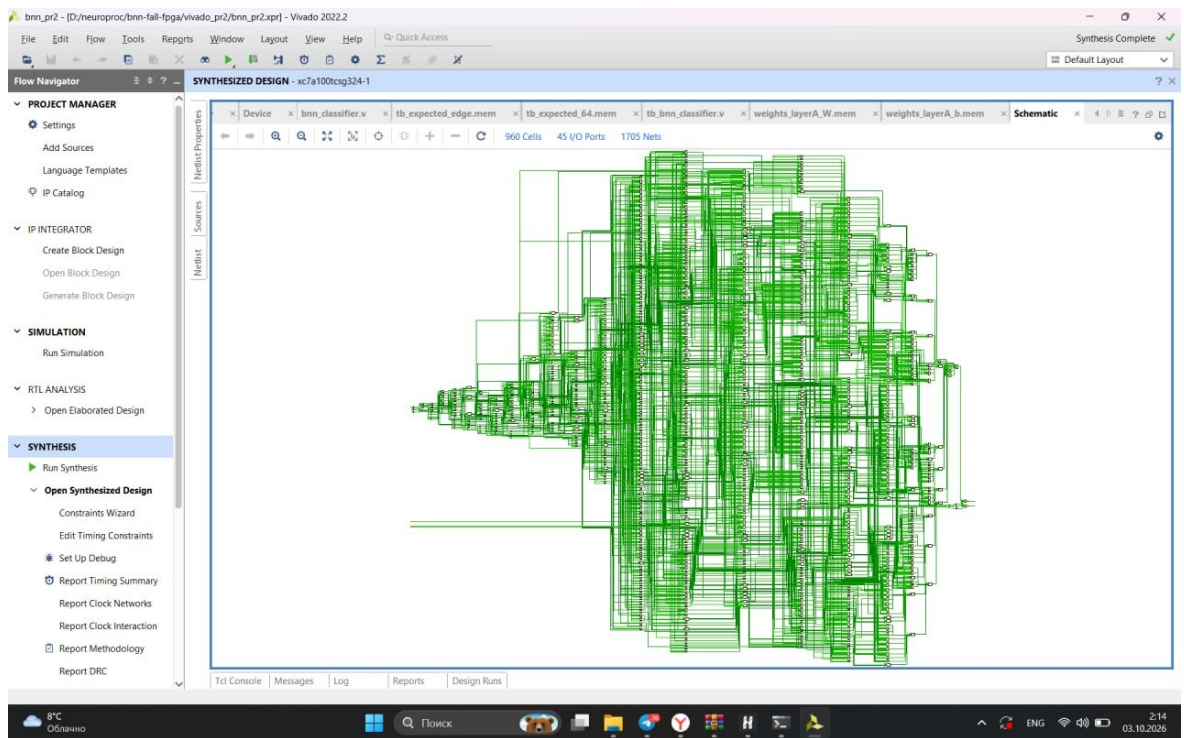


Рисунок В.4 — Общий вид синтезированной логической схемы

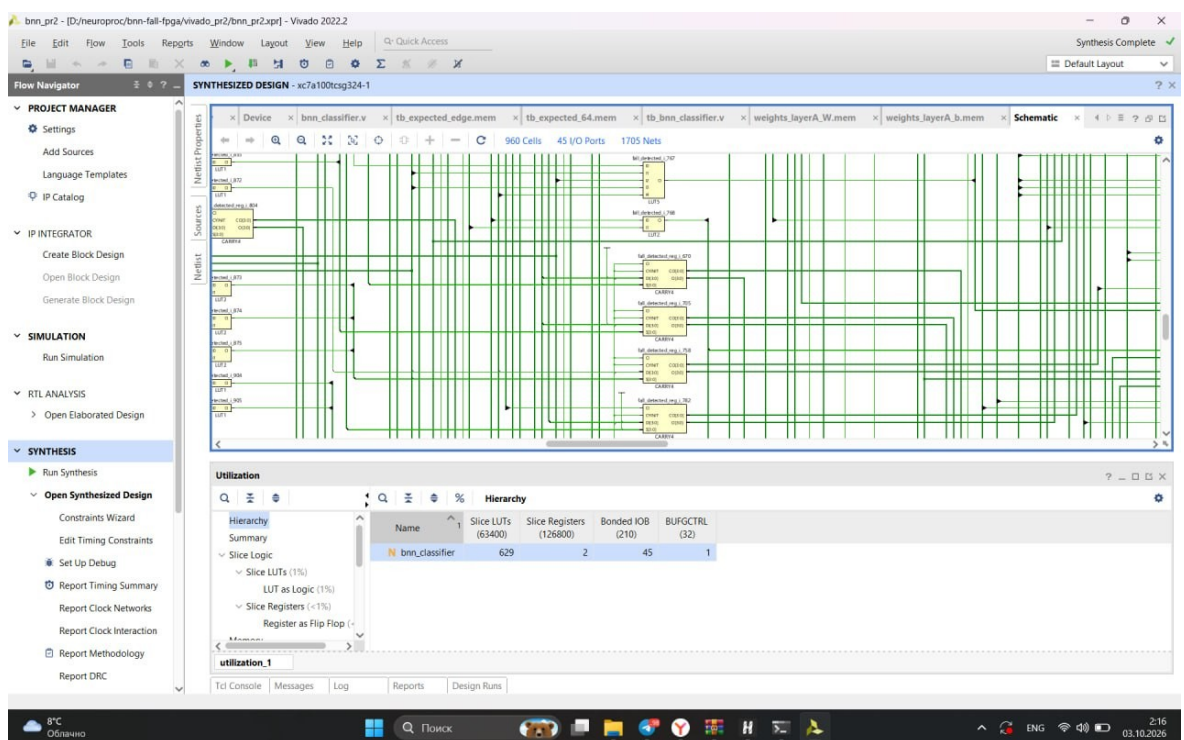


Рисунок В.5 — Фрагмент схемы и использование ресурсов после синтеза